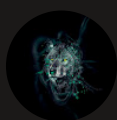
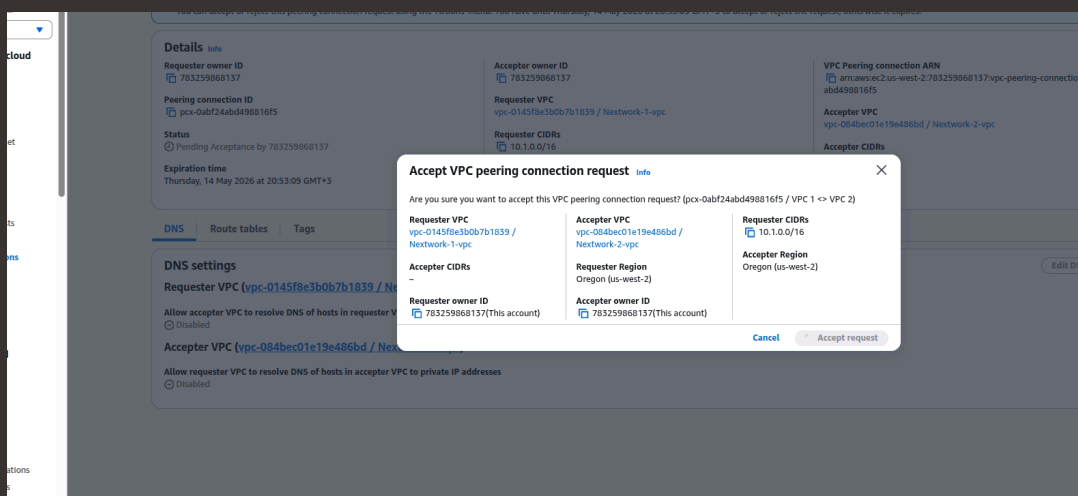


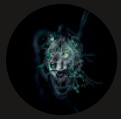


VPC Peering



Kiprono Yegon





Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service from Amazon Web Services that allows you to create a private, isolated network within the cloud where you can securely run your applications and resources. It is useful because it gives you full control over your networking environment, including IP address ranges, subnets, routing, and security settings, enabling you to protect your systems, manage traffic flow, and design scalable architectures.

How I used Amazon VPC in this project

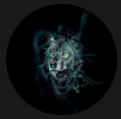
I used Amazon VPC to create different cloud resources that isolated the resources according to the needs I had

One thing I didn't expect in this project was...

One thing I didn't expect in this project was the simplicity in setting up VPC peering

This project took me...

This project took me about forty five minutes give or take



In the first part of my project...

Step 1 - Set up my VPC

In this step, I will crate two VPCs from scratch whilst using the VPC resource map to create the VPCs easily

Step 2 - Create a Peering Connection

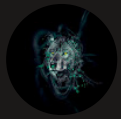
In this step, I will create a connection between the two VPCs I had launched because it will enable the two VPCs to share the resources without using the internet

Step 3 - Update Route Tables

In this step, I will update the route table; by setting a way for traffic coming from VPC 1 to get to VPC 2.because this enables the VPCs to share resources to and fro each VPC

Step 4 - Launch EC2 Instances

In this step, I will launch an EC2 instance in each VPC because we will use them to test your VPC peering connection



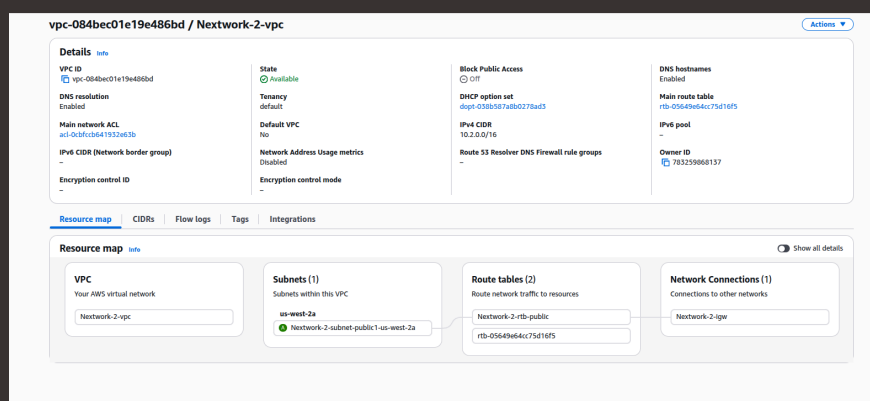
Multi-VPC Architecture

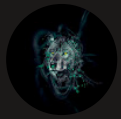
I started my project by launching the first VPC using the resource map. I launched the VPC using only one subnet-public subnet only

The CIDR blocks for VPCs 1 and 2 are different. They have to be unique because having the same IP addresses will have the resources overlapping. Having overlapping IP addresses could cause routing conflicts and connectivity issues!

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances as we will be using EC2 Instance Connect and AWS actually manages a key pair for us! We don't need to manage key pairs ourselves.





VPC Peering

A VPC peering connection is a direct connection between two VPCs. A peering connection lets VPCs and their resources route traffic between them using their private IP addresses. This means data can now be transferred between VPCs without going through the public internet.

VPCs would use peering connections to route traffic of their resources using private IP addresses without using the internet.

The difference between a Requester and an Acceptor in a peering connection is the Requester is the VPC that initiates a peering connection. As the requester, they will be sending the other VPC an invitation to connect and the Acceptor is the VPC that receives a peering connection request! The Acceptor can either accept or decline the invitation

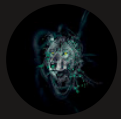
The screenshot shows the AWS VPC console interface for creating a VPC peering connection. The title bar indicates 'VPC 1 ↔ VPC 2'. The main section is titled 'Select a local VPC to peer with'. Under 'VPC ID (Requester)', a dropdown menu shows 'vpc-0145f8e3b0b7b1839 (Nextwork-1-vpc)'. Below this, a table lists VPC CIDRs for the requester VPC:

CIDR	Status	Status reason
10.1.0.0/16	Associated	-

The next section is 'Select another VPC to peer with'. It includes radio buttons for 'My account' (selected) and 'Another account'. Below, there are radio buttons for 'This Region (us-west-2)' (selected) and 'Another Region'. Under 'VPC ID (Acceptor)', a dropdown menu shows 'vpc-0b48ec01e19e408bd (Nextwork-2-vpc)'. Below this, a table lists VPC CIDRs for the acceptor VPC:

CIDR	Status	Status reason
10.2.0.0/16	Associated	-

At the bottom, there is a 'Tags' section with a description: 'A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.' Below this is a 'Key' field with a search icon and a dropdown menu showing 'Q, VPC 1 ↔ VPC 2'. There is also a 'Remove' button.



Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because traffic in VPC 1 won't know how to get to resources in VPC 2 without a route in the route table!

My VPCs' new routes have a destination of the VPC address of the latter VPC. The routes' target was the peering connection we had created earlier.

Route table ID rtb-0240af0c328ca13b8	Main No	Explicit subnet associations subnet-00668c8d8d5d17e50 / Nextwork-2-subnet-public1-us-west-2a	Edge associations -
VPC vpc-084bec01e19e486bd Nextwork-2-vpc	Owner ID 783259868137		

Routes

Subnet associations

Edge associations

Route propagation

Tags

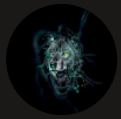
Routes (3)

Filter routes

Both

Edit routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	lgw-0cee271a0fa1ad58c	Active	No	Create Route
10.1.0.0/16	pcx-0abf24abd498816f5	Active	No	Create Route
10.2.0.0/16	local	Active	No	Create Route Table



In the second part of my project...

Step 5 - Use EC2 Instance Connect

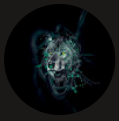
In this step, I will use EC2 Instance Connect to connect to the first Instance and try fixing any error before trying to set a linkage

Step 6 - Connect to EC2 Instance 1

In this step, I will use the EC2 Connect to connect to the Instance with the first error solved.

Step 7 - Test VPC Peering

In this step, I will get instance 1 to send test messages to instance 2, then solve errors in Instance to until it can send messages back



Troubleshooting Instance Connect

Next, I used EC2 Instance Connect to connect to the Instance because we did not use the key-pair as Amazon handles the key pair

I was stopped from using EC2 Instance Connect as the Instance did not have a public IP address which is necessary when using EC2 Instance Connect to connect to the instance

Connect info
Connect to an instance using the browser-based client.

Instance ID i-06cfadc510f3d21cd (Instance - NextWork VPC 1)	VPC ID vpc-0145f8e3b0b7b1839	Security groups sg-0500627500c74ca78 (default)	IAM role -
---	--	--	----------------------

EC2 Instance Connect | SSM Session Manager | SSH client | EC2 serial console

No public IPv4 or IPv6 address assigned
With no public IPv4 or IPv6 address, you can't use EC2 Instance Connect. Alternatively, you can try connecting using [EC2 Instance Connect Endpoint](#).

Instance ID
i-06cfadc510f3d21cd (Instance - NextWork VPC 1)

Connection type

☒ **Connect using a Public IP**
Connect using a public IPv4 or IPv6 address

☐ **Connect using a Private IP**
Connect using a private IP address and a VPC endpoint

☐ **Public IPv4 address**

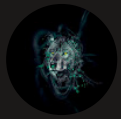
☐ **IPv6 address**

Username
Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user.

ec2-user

Note: In most cases, the default username, ec2-user, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

[Cancel](#) [Connect](#)



Elastic IP addresses

To resolve this error, I set up Elastic IP addresses. Elastic IP addresses are static IPv4 addresses that get allocated to your AWS account, and is yours to delegate to an EC2 instance.

Associating an Elastic IP address resolved the error because it gives the instance a public IPv4 address that is necessary when using EC2 Instance Connect to connect to the instance

Allocate Elastic IP address [Info](#)

Elastic IP address settings [Info](#)

Public IPv4 address pool

- ☒ Amazon's pool of IPv4 addresses
 - ☐ Public IPv4 address that you bring to your AWS account with BYOIP. (option disabled because no pools found) [Learn more](#)
 - ☐ Customer-owned pool of IPv4 addresses created from your on-premises network for use with an Outpost. (option disabled because no customer owned pools found) [Learn more](#)
 - ☐ Allocate using an IPv4 IPAM pool (option disabled because no public IPv4 IPAM pools with AWS service as EC2 were found)

Network border group [Info](#)

us-west-2

Global static IP addresses

AWS Global Accelerator can provide global static IP addresses that are announced worldwide using anycast from AWS edge locations. This can help improve the availability and latency for your user traffic by using the Amazon global network. [Learn more](#)

[Create accelerator](#)

Tags - optional

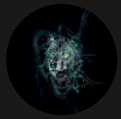
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tag

[Cancel](#) [Allocate](#)



Troubleshooting ping issues

To test VPC peering, I ran the command ping and the private IPv4 address of the second instance

A successful ping test would validate my VPC peering connection because it would return new lines on the terminal

I had to update my second EC2 instance's security group because it's security group does not allow inbound from the first instance. I added a new rule that allows inbound traffic from the first instance

```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-1-1-55 ~]$ cat /etc/os-release
cat: /etc/os-release: No such file or directory
[ec2-user@ip-10-1-1-55 ~]$ cat /etc/os-release
NAME="Amazon Linux"
VERSION="2023"
ID="amzn"
ID_LIKE="fedora"
VERSION_ID="2023"
PLATFORM_ID="platform:al2023"
PRETTY_NAME="Amazon Linux 2023.11.20260505"
ANSI_COLOR="0;32"
CPE_NAME="cpe:2.3:amazon:amazon_linux:2023"
HOME_URL="https://aws.amazon.com/linux/amazon-linux-2023/"
DOCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
SUPPORT_URL="https://aws.amazon.com/premiumsupport/"
BUG_REPORT_URL="https://github.com/amazonlinux/amazon-linux-2023"
VENDOR_NAME="AMZ"
VENDOR_URL="https://aws.amazon.com/"
SUPPORT_EMAIL="2023-06-50"

[ec2-user@ip-10-1-1-55 ~]$ ping 10.2.3.4
PING 10.2.3.4 (10.2.3.4) 56(84) bytes of data:
64 bytes from 10.2.3.4: icmp_seq=259 ttl=127 time=1.43 ms
64 bytes from 10.2.3.4: icmp_seq=260 ttl=127 time=0.559 ms
64 bytes from 10.2.3.4: icmp_seq=261 ttl=127 time=0.576 ms
64 bytes from 10.2.3.4: icmp_seq=262 ttl=127 time=0.609 ms
64 bytes from 10.2.3.4: icmp_seq=263 ttl=127 time=0.800 ms
64 bytes from 10.2.3.4: icmp_seq=264 ttl=127 time=1.11 ms
64 bytes from 10.2.3.4: icmp_seq=265 ttl=127 time=1.07 ms
64 bytes from 10.2.3.4: icmp_seq=266 ttl=127 time=0.333 ms
64 bytes from 10.2.3.4: icmp_seq=267 ttl=127 time=0.348 ms
^C
--- 10.2.3.4 ping statistics ---
267 packets transmitted, 9 received, 96.6292% packet loss, time 276529ms
rtt min/avg/max/mdev = 0.333/0.769/1.427/0.353 ms
[ec2-user@ip-10-1-1-55 ~]$
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

